

Assignment 1

Task 1.1 Calculate the Clock Granularity of different Python Timers

We start with defining the given "checktick"-function to calculate the clock granularity of the "time.time()" clock.

```
In [27]: from timeit import default_timer as timer
import numpy as np
import time

def checktick():
    M = 200
    timesfound = np.empty((M,))
    for i in range(M):
        t1 = time.time() # get timestamp from timer
        t2 = time.time() # get timestamp from timer
        while (t2 - t1) < 1e-16: # if zero then we are below clock granularity, retake timing
            t2 = time.time() # get timestamp from timer
        t1 = t2 # this is outside the loop
        timesfound[i] = t1 # record the time stamp
    minDelta = 1000000
    Delta = np.diff(timesfound) # it should be cast to int only when needed
    minDelta = Delta.min() * 1e9 # convert to nanoseconds
    return minDelta

print("Clock granularity of time.time() is ", checktick(), " nanoseconds")

Clock granularity of time.time() is 715.2557373046875 nanoseconds

Now we modify the "checktick"-function to calculate the clock granularity of "timeit".
```

```
In [28]: def checktick():
    M = 200
    timesfound = np.empty((M,))
    for i in range(M):
        t1 = timer() # get timestamp from timer
        t2 = timer() # get timestamp from timer
        while (t2 - t1) < 1e-16: # if zero then we are below clock granularity, retake timing
            t2 = timer() # get timestamp from timer
        t1 = t2 # this is outside the loop
        timesfound[i] = t1 # record the time stamp
    minDelta = 1000000
    Delta = np.diff(timesfound) # it should be cast to int only when needed
    minDelta = Delta.min() * 1e9 # convert to nanoseconds
    return minDelta

print("Clock granularity of timeit is ", checktick(), " nanoseconds")

Clock granularity of timeit is 165.9936970998068 nanoseconds

Now we modify it for the "time.time_ns" timer:
```

```
In [29]: def checktick():
    M = 200
    timesfound = np.empty((M,))
    for i in range(M):
        t1 = time.time_ns() # get timestamp from timer
        t2 = time.time_ns() # get timestamp from timer
        while (t2 - t1) < 1e-16: # if zero then we are below clock granularity, retake timing
            t2 = time.time_ns() # get timestamp from timer
        t1 = t2 # this is outside the loop
        timesfound[i] = t1 # record the time stamp
    minDelta = 1000000
    Delta = np.diff(timesfound) # it should be cast to int only when needed
    minDelta = Delta.min()
    return minDelta

print("Clock granularity of time.time_ns is ", checktick(), " nanoseconds")

Clock granularity of time.time_ns is 768.0 nanoseconds

The results show, that the "timeit" timer has the smallest clock granularity.
```

Task 1.2 Timing the Julia set code functions.

We modified the given decorator and put it in a new file, to keep the original one. The new code is in the file named "JuliaSet-timing".

Changes made to the decorator:

- Change the timer to the one from timeit, because it turned out to have the smallest clock granularity.
- Run Code multiple times (5 times), to get average and standard deviation data.

Modified code of the decorator:

```
def timefn(fn):
    @wraps(fn)
    def measure_time(*args, **kwargs):
        n = 5
        times = []

        for i in range(n):
            t1 = timer()
            result = fn(*args, **kwargs)
            t2 = timer()
            times.append(t2 - t1)
        avg = np.mean(times)
        std = np.std(times)
        print(f"@timefn: {fn.__name__} took (avg:{.6f} seconds, standard deviation: {std:.6f}) (n={n})")
        return result
    return measure_time

# Since the "calc_pure_python" function calls the "calculate_z_serial_purepython" inside itself, we first only timed the "calc_pure_python" function (in JuliaSet-timing.py).
```

```
In [30]: %run JuliaSet-timing.py

Length of x: 1000
Total elements: 1000000
Length of x: 1000
Total elements: 1000000
Length of x: 1000
Total elements: 1000000
Length of x: 1000
Total elements: 1000000
Length of x: 1000
Total elements: 1000000
@timefn: calc_pure_python took 3.963043 seconds, standard deviation: 0.042175 (n=5)

Now the function "calculate_z_serial_purepython" is timed, for that we use the exact same code as before but only with the decorator used on "calculate_z_serial_purepython". This is done in a separate file
"JuliaSet-timing2.py".
```

```
In [31]: %run JuliaSet-timing2.py

Length of x: 1000
Total elements: 1000000
@timefn: calculate_z_serial_purepython took 3.647499 seconds, standard deviation: 0.026005 (n=5)

Since the standard deviation is far bigger than the clock granularity, the timer function is not limited by the clock granularity. If it was the same value as the clock granularity, this would mean that the timer is not
fast enough and the results depend on the timer itself.
```

Task 1.3 Profile the Julia set code with cProfile and line_profiler the computation.

Since "cProfile" doesn't need changes in the code we can use the original code in the file "JuliaSet-original.py". We will time the execution with the profilers to analyze the overhead afterwards.

```
In [32]: t1 = timer()
ipython -m cProfile -s cumulative JuliaSet-original.py
t2 = timer()
time_cProfiler = t2 - t1
print("Execution time with cProfile: ", time_cProfiler, " seconds")

Length of x: 1000
Total elements: 1000000
calculate_z_serial_purepython took 4.953953981399536 seconds
36221905 function calls in 5.266 seconds

Ordered by: cumulative time

ncalls  tottime  percall  ctime    percall  filename:lineno(function)
1 0.000 0.000 0.000 0.000 5.266 {built-in method builtins.exec}
1 0.011 0.011 0.011 0.011 5.266 JuliaSet-original.py:1(<module>)
1 0.250 0.250 0.255 0.255 5.255 JuliaSet-original.py:21(calc_pure_python)
4 4.005 4.005 4.954 4.954 JuliaSet-original.py:59(calculate_z_serial_purepython)
34219000 0.040 0.000 0.040 0.000 {built-in method builtins.abs}
2002000 0.004 0.000 0.004 0.000 {method 'append' of 'list' objects}
1 0.004 0.004 0.004 0.004 {built-in method builtins.sum}
3 0.000 0.000 0.000 0.000 {built-in method builtins.print}
2 0.000 0.000 0.000 0.000 {built-in method time.time}
4 0.000 0.000 0.000 0.000 {built-in method builtins.len}
1 0.000 0.000 0.000 0.000 {method 'disable' of '_lsprof.Profiler' objects}
```

Execution time with cProfile: 5.586678041004254 seconds

The results tell us that we spend a lot of time in the calculate_z_serial_purepython function. There the abs() function is called many times, which takes some time. Now let's save it to a file so we can visualize the results with snakeviz afterwards.

```
In [33]: !python -m cProfile -o profile.stats JuliaSet-original.py

Length of x: 1000
Total elements: 1000000
calculate_z_serial_purepython took 4.923141002655029 seconds
```

```
In [34]: !python -m snakeviz profile.stats --server

snakeviz web server started on 127.0.0.1:8080; enter Ctrl-C to exit
http://127.0.0.1:8080/snakeviz?2fUsers%2Ftimeberle%2FgitHub%2FHPC-Intro%2Fprofile.stats
^C

First we will use the lineProfiler for the first function "calc_pure_python". Therefore we need to use the decorator only on this function, which is done in the file "JuliaSet-lineProfiler.py".
```

```
In [35]: t1 = timer()
ipython -m kernProf -l JuliaSet-lineProfiler.py
t2 = timer()
ipython -m line_profiler JuliaSet-lineProfiler.py.lprof
time_lineProfiler1 = t2 - t1
print("Execution time with lineProfiler on calc_pure_python: ", time_lineProfiler1, " seconds")

Length of x: 1000
Total elements: 1000000
calculate_z_serial_purepython took 14.613469123840332 seconds
Wrote profile results to 'JuliaSet-lineProfiler.py.lprof'
Inspect results with:
python -m line_profiler -rmt JuliaSet-lineProfiler.py.lprof
Timer unit: 1e-06 s

Total time: 15.3213 s
File: JuliaSet-lineProfiler.py
Function: calc_pure_python at line 20

Line #      Hits         Time  Per Hit   % Time  Line Contents
=====
20
21                                     @profile
22 def calc_pure_python(desired_width, max_iterations):
23     """Create a list of complex coordinates (zs) and complex parameters (cs),
24     build Julia set"""
25     x_step = (x2 - x1) / desired_width
26     y_step = (y1 - y2) / desired_width
27     x = []
28     y = []
29     ycoord = y2
30     while ycoord > y1:
31         y.append(ycoord)
32         ycoord += y_step
33     xcoord = x1
34     while xcoord < x2:
35         x.append(xcoord)
36         xcoord += x_step
37     # build a list of coordinates and the initial condition for each cell.
38     # Note that our initial condition is a constant and could easily be removed,
39     # we use it to simulate a real-world scenario with several inputs to our
40     # function
41     zs = []
42     cs = []
43     for ycoord in y:
44         for xcoord in x:
45             zs.append(complex(xcoord, ycoord))
46             cs.append(complex(c_real, c_imag))
47
48     print("Length of x:", len(x))
49     print("Total elements:", len(zs))
50     start_time = time.time()
51     output = calculate_z_serial_purepython(max_iterations, zs, cs)
52     end_time = time.time()
53     secs = end_time - start_time
54     print(calculate_z_serial_purepython.__name__ + " took", secs, "seconds")
55
56     # This sum is expected for a 1000^2 grid with 300 iterations
57     # It ensures that our code evolves exactly as we'd intended
58     assert sum(output) == 33219900
```

Execution time with lineProfiler on calc_pure_python: 16.037393958002212 seconds

The lineProfiler tells us how many time each line takes. Here the function call of "calculate_z_serial_purepython" takes most of the time.

Now let's do it also for the function "calculate_z_serial_purepython", in file "JuliaSet-lineProfiler2.py".

```
In [36]: t1 = timer()
ipython -m kernProf -l JuliaSet-lineProfiler2.py
t2 = timer()
ipython -m line_profiler JuliaSet-lineProfiler2.py.lprof
time_lineProfiler2 = t2 - t1
print("Execution time with lineProfiler on calculate_z_serial_purepython: ", time_lineProfiler2, " seconds")

Length of x: 1000
Total elements: 1000000
calculate_z_serial_purepython took 31.534297943115234 seconds
Wrote profile results to 'JuliaSet-lineProfiler2.py.lprof'
Inspect results with:
python -m line_profiler -rmt JuliaSet-lineProfiler2.py.lprof
Timer unit: 1e-06 s

Total time: 19.8073 s
File: JuliaSet-lineProfiler2.py
Function: calculate_z_serial_purepython at line 58

Line #      Hits         Time  Per Hit   % Time  Line Contents
=====
58
59                                     @profile
60 def calculate_z_serial_purepython(maxiter, zs, cs):
61     """Calculate output list using Julia update rule"""
62     output = [0] * len(zs)
63     for i in range(len(zs)):
64         n = 0
65         z = zs[i]
66         c = cs[i]
67         while abs(z) < 2 and n < maxiter:
68             z = z * z + c
69             n += 1
70         output[i] = n
71     return output

Execution time with lineProfiler on calculate_z_serial_purepython: 32.14328783300152 seconds
```

The while loop here takes the most time, since it is called many times.

To analyze the overhead of the profilers we will compare the time it takes to run the code with and without the profilers. Therefore we will run it once without the profilers.

```
In [37]: t1 = timer()
%run JuliaSet-original.py
t2 = timer()
time_noProfiler = t2 - t1
print("Execution time without profiler: ", time_noProfiler, " seconds")
overhead_cProfiler = time_cProfiler - time_noProfiler
overhead_lineProfiler1 = time_lineProfiler1 - time_noProfiler
overhead_lineProfiler2 = time_lineProfiler2 - time_noProfiler
print("Overhead of cProfile: ", overhead_cProfiler, " seconds")
print("Overhead of lineProfiler on calc_pure_python: ", overhead_lineProfiler1, " seconds")
print("Overhead of lineProfiler on calculate_z_serial_purepython: ", overhead_lineProfiler2, " seconds")

Length of x: 1000
Total elements: 1000000
calculate_z_serial_purepython took 3.640277093914795 seconds
Execution time without profiler: 3.865267250002944 seconds
Overhead of cProfile: 1.72140279100131 seconds
Overhead of lineProfiler on calc_pure_python: 12.172126707999269 seconds
Overhead of lineProfiler on calculate_z_serial_purepython: 28.2780205299958 seconds
```

Task 1.4 Memory-profile the JuliaSet code. Use the memory_profiler and mprof to profile the computation in JuliaSet code.

Now we will profile the memory usage of the JuliaSet code. We start with using the memory_profiler on the two functions. We will also time the execution times to measure the overhead afterwards. Since it needs the same decorator as the lineprofiler we can use the same files for it.

```
In [38]: t1 = timer()
ipython -m memory_profiler JuliaSet-lineProfiler.py
t2 = timer()
time_memory_profiler1 = t2 - t1
print("Execution time with memory_profiler on calc_pure_python: ", time_memory_profiler1, " seconds")
t1 = timer()
ipython -m memory_profiler JuliaSet-lineProfiler2.py
t2 = timer()
time_memory_profiler2 = t2 - t1
print("Execution time with memory_profiler on calculate_z_serial_purepython: ", time_memory_profiler2, " seconds")

Length of x: 1000
Total elements: 1000000
calculate_z_serial_purepython took 47.21134901046753 seconds
Filename: JuliaSet-lineProfiler.py

Line #      Mem usage      Increment  Occurrences  Line Contents
=====
20
21                                     @profile
22 def calc_pure_python(desired_width, max_iterations):
23     """Create a list of complex coordinates (zs) and complex parameters (cs),
24     build Julia set"""
25     x_step = (x2 - x1) / desired_width
26     y_step = (y1 - y2) / desired_width
27     x = []
28     y = []
29     ycoord = y2
30     while ycoord > y1:
31         y.append(ycoord)
32         ycoord += y_step
33     xcoord = x1
34     while xcoord < x2:
35         x.append(xcoord)
36         xcoord += x_step
37     # build a list of coordinates and the initial condition for each cell.
38     # Note that our initial condition is a constant and could easily be removed,
39     # we use it to simulate a real-world scenario with several inputs to our
40     # function
41     zs = []
42     cs = []
43     for ycoord in y:
44         for xcoord in x:
45             zs.append(complex(xcoord, ycoord))
46             cs.append(complex(c_real, c_imag))
47
48     print("Length of x:", len(x))
49     print("Total elements:", len(zs))
50     start_time = time.time()
51     output = calculate_z_serial_purepython(max_iterations, zs, cs)
52     end_time = time.time()
53     secs = end_time - start_time
54     print(calculate_z_serial_purepython.__name__ + " took", secs, "seconds")
55
56     # This sum is expected for a 1000^2 grid with 300 iterations
57     # It ensures that our code evolves exactly as we'd intended
58     assert sum(output) == 33219900

Execution time with memory_profiler on calc_pure_python: 94.26334858300106 seconds

Length of x: 1000
Total elements: 1000000
calculate_z_serial_purepython took 1584.7740099430084 seconds
Filename: JuliaSet-lineProfiler2.py

Line #      Mem usage      Increment  Occurrences  Line Contents
=====
58 126.312 MiB  126.312 MiB  1 @profile
59 def calculate_z_serial_purepython(maxiter, zs, cs):
60     """Calculate output list using Julia update rule"""
61     output = [0] * len(zs)
62     for i in range(len(zs)):
63         n = 0
64         z = zs[i]
65         c = cs[i]
66         while abs(z) < 2 and n < maxiter:
67             z = z * z + c
68             n += 1
69             output[i] = n
70     return output

Execution time with memory_profiler on calculate_z_serial_purepython: 1585.628453833997 seconds
```

We can see, that the memory usage increments the most when the list "zs" and "cs" are filled.

Let's analyze the code by using the mprof.

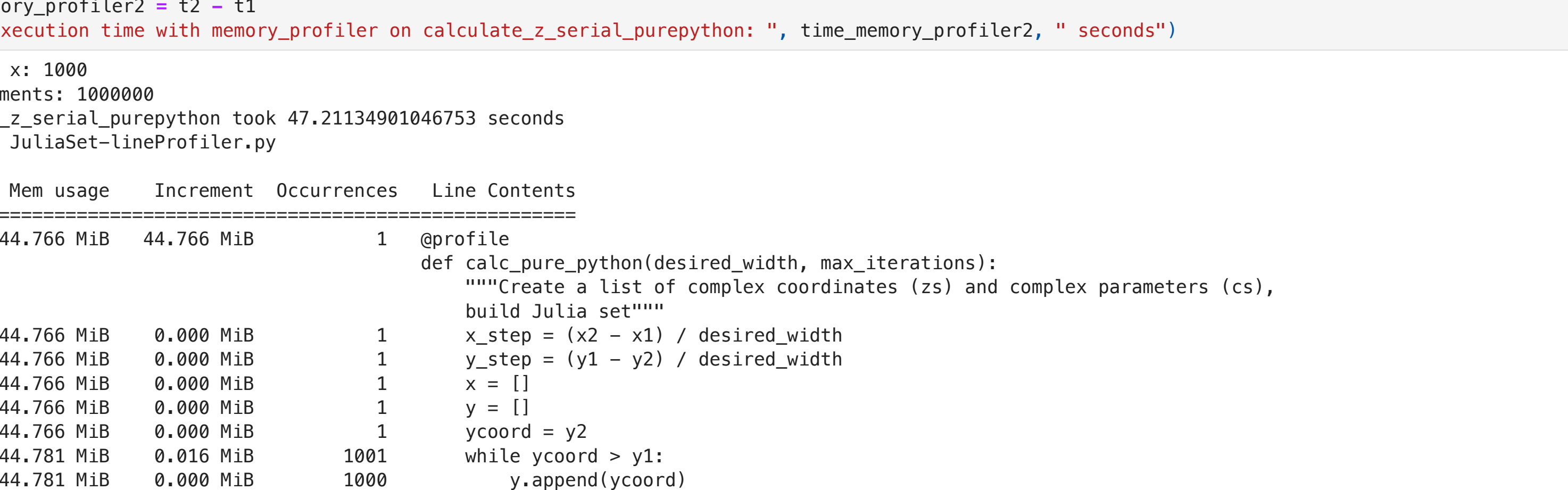
```
In [44]: t1 = timer()
ipython -m mprof run JuliaSet-lineProfiler.py
t2 = timer()
time_mprof1 = t2 - t1
print("Execution time with mprof on calc_pure_python: ", time_mprof1, " seconds")
t1 = timer()
ipython -m mprof run JuliaSet-lineProfiler2.py
t2 = timer()
time_mprof2 = t2 - t1
print("Execution time with mprof on calculate_z_serial_purepython: ", time_mprof2, " seconds")

mprof.py: Sampling memory every 0.1s
running new process
running as a Python program...
Length of x: 1000
Total elements: 1000000
calculate_z_serial_purepython took 3.6699697971343994 seconds
Execution time with mprof on calc_pure_python: 4.858620660083844 seconds
mprof.py: Sampling memory every 0.1s
running new process
running as a Python program...
Length of x: 1000
Total elements: 1000000
calculate_z_serial_purepython took 3.71606183052063 seconds
Execution time with mprof on calculate_z_serial_purepython: 4.78319695800019 seconds
```

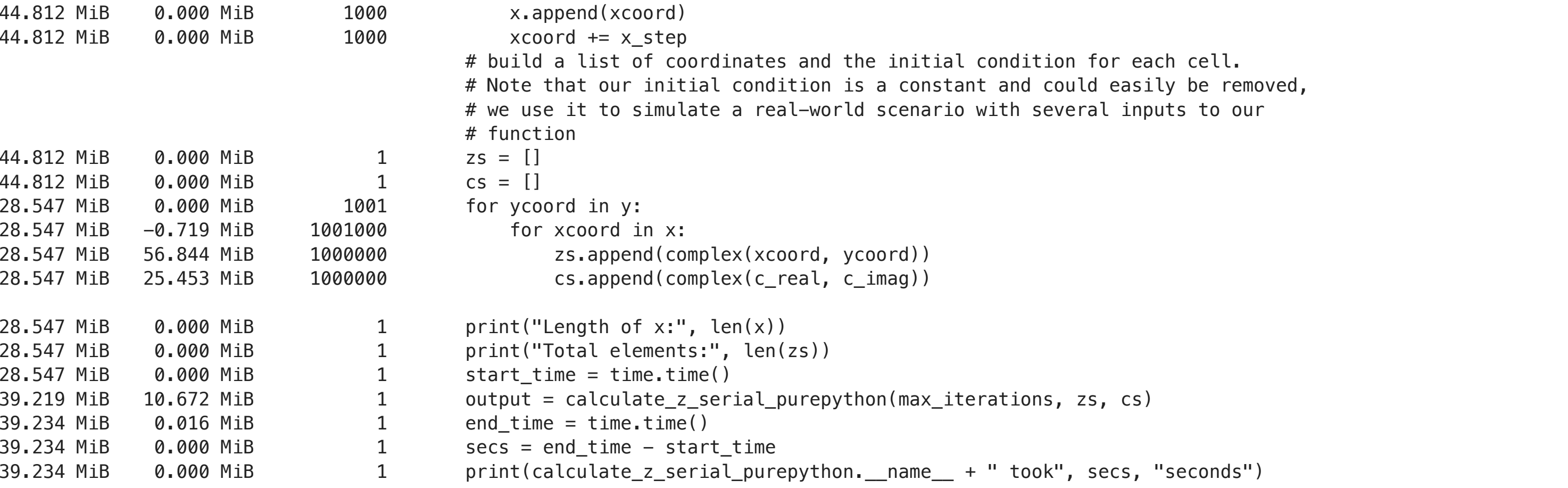
The mprof plots can't be displayed when executing in Jupyter Notebook. The following two images are the resulting plots, first of only the calc_pure_python, second of calculate_z_serial_purepython function.

The memory usage initially ramps up quite fast and then grows slow and steady over time.

/Users/timeberle/HPC-Intro/.venv/bin/python JuliaSet-lineProfiler.py



/Users/timeberle/GitHub/HPC-Intro/.venv/bin/python JuliaSet-lineProfiler2.py



```
In [45]: overhead_mprof1 = time_mprof1 - time_noProfiler
overhead_mprof2 = time_mprof2 - time_noProfiler
print("Overhead of mprof on calc_pure_python: ", overhead_mprof1, " seconds")
print("Overhead of mprof on calculate_z_serial_purepython: ", overhead_mprof2, " seconds")

Overhead of mprof on calc_pure_python: 0.993534160009003 seconds
Overhead of mprof on calculate_z_serial_purepython: 0.9179257079972457 seconds
```

```
In [46]:
```